

Kernal

A kernel is the core part of an operating system. It acts as a bridge between software applications and the hardware of a computer.

The kernel manages system resources, such as the CPU, memory and devices, ensuring everything works together smoothly and efficiently.

It handles tasks like running programs, accessing files and connecting to devices like printers and keyboards.

An Operating System includes the kernel as its core, but also provides a user interface, file system management, network services and various utility applications that allow users to interact with the system

Types of Kernel

The following are different types of kernels.

Monolithic kernel: all OS services run in kernel space → fast, but less fault-isolation. Examples are Unix, Linux, Open VMS, XTS-400 etc.

Microkernel: minimal kernel functionality; most services moved to user space → better reliability, but more overhead. Examples are Minix 3 and Mach (true microkernel versions like Mach 3.0),

Hybrid kernel: mixes monolithic + microkernel ideas; some services in kernel for speed, others isolated for safety. Examples are Windows NT family (Windows 2000, XP, Vista, 7, 8, 10 etc.), macOS / XNU, ReactOS and Haiku OS

Nanokernel: extremely minimal kernel, providing only basic hardware abstraction; everything else outside. Examples are Nemesis and MIT Exokernel projects like XOK, Aegis

Exokernel: separates protection vs. manage

Modern operating systems use a dual-mode architecture to protect system integrity and hardware. The CPU operates in two different privilege levels:

1. User Mode

The restricted environment where applications and user programs run.

Privileges:

- Cannot directly access hardware or memory
- Limited instruction set available
- Can only access its own allocated memory space

Purpose:

- Prevents user applications from crashing the system
- Protects system resources from unauthorized access
- Examples: Web browsers, word processors, games, your custom applications

What happens in User Mode:

Your App → Requests system resource → System Call → Switch to Kernel Mode

2. Kernel Mode (Supervisor/Privileged Mode)

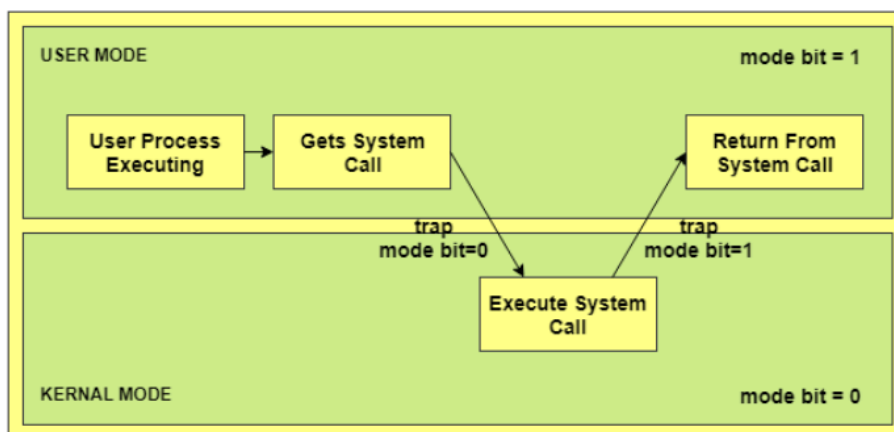
The privileged environment where the operating system core runs.

Privileges:

- Full access to hardware and all memory
- Can execute any CPU instruction
- Direct hardware control

Purpose:

- Manages system resources
- Handles hardware devices
- Provides essential services to user programs
- **Examples:** Device drivers, memory management, process scheduling, file systems



Key Differences

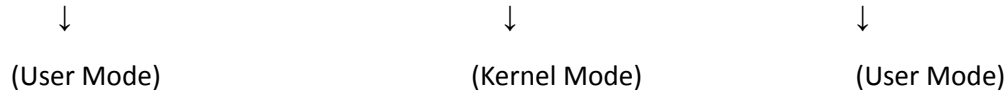
Aspect	User Mode	Kernel Mode
Access	Limited access to memory/hardware	Complete access to everything
Crash Impact	Only crashes the application	Can crash entire system
Privilege Level	Low privilege (Ring 3 on x86)	High privilege (Ring 0 on x86)
Operations	Normal application code	Critical OS operations

How They Work Together

- User program needs hardware resource (e.g., read a file)
- Program issues a system call (software interrupt)

- Mode switch occurs (User → Kernel)
- Kernel performs the privileged operation
- Mode switch back (Kernel → User)
- Result returned to user program

User Program → System Call → SWITCH → Kernel → Execute → SWITCH → User Program



Real-World Example:

Think of a restaurant:

- User Mode = Customer: Can order food, sit at tables, but can't enter kitchen
- Kernel Mode = Chef: Has full access to kitchen, equipment, and ingredients
- System Call = Order via waiter: Customer's request that gets passed to kitchen

Importance

- **Security:** Prevents malicious programs from taking over system
- **Stability:** User program crashes don't affect OS or other programs
- **Resource Management:** OS controls fair allocation of CPU, memory, etc.
- This separation is fundamental to modern computing, ensuring that even if your application has a bug or is malicious, it cannot compromise the entire operating system.

System Call

- The raw service request to the OS kernel
- What: A program's direct request for OS service
- Level: Lowest level (kernel mode)
- Example: `sys_read`, `sys_write` (Linux), `NtReadFile` (Windows)
- Who uses it: OS itself and system libraries

System Call Interface (SCI)

- The official door to request OS services
- What: The complete set of available system calls
- Level: Boundary between user space and kernel space
- Example: Linux syscall table (with numbers 0, 1, 2... for each call)
- Who uses it: Libraries and language runtimes

API (Application Programming Interface)

- The friendly menu for programmers
- What: High-level functions that may use system calls
- Level: Highest level (user space)
- Example: `fopen()`, `printf()` (C), `CreateWindow()` (Win32)

- Who uses it: Programmers like you!

